

## СӨЖ №2 есебі

**Тақырыбы:** «Онлайн сауда платформасы (E-commerce) ДҚ жобалау – Нормалды формалар, Процедуралар, Триггерлер және Транзакциялар»

**Орындаған:** Студент Тауан Мейіржан Олжасұлы

### Кіріспе

Бұл жұмыстың мақсаты – «Онлайн сауда платформасы» (E-commerce) үшін жобаланған дерекқорды нормализациялау, оның тұтастығын сақтау мақсатында процедуралар мен триггерлер жазу, сондай-ақ ACID қағидаларына негізделген транзакцияларды басқару. Есеп барысында PostgreSQL (pgAdmin) ортасында DDL және DML операциялары орындалып, тестіленді.

### 1.

ДҚ-ны нормализациялау есебі

Бастапқы деректер 1NF, 2NF және 3NF талаптарына келтірілді. Ішінара және транзитивті тәуелділіктерді жою мақсатында деректер Customers, Products, Orders және OrderItems кестелеріне бөлінді. Бұл құрылым автоматты түрде BCNF формасына жауап береді.

Кестелер құрылатын бөлімге түсіндірме:

"Дерекқорды жобалау кезінде мәліметтердің қайталануын (аномалияларды) болдырмау үшін нормализация жасадым. Тауарлар (Products), клиенттер (Customers) және олардың тапсырыстары (Orders, OrderItems) бөлек кестелерге бөлінді. Бұл құрылым 3NF және BCNF формаларының талаптарына толық жауап береді. Сонымен қатар, REFERENCES арқылы кестелер арасындағы сыртқы кілт (Foreign Key) байланыстарын орнаттым."

**Кестелерді құру және бастапқы деректерді енгізу (DDL & INSERT):**

SQL

```
CREATE TABLE Customers (  
    customer_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    email VARCHAR(100) UNIQUE NOT NULL  
);  
  
CREATE TABLE Products (  
    product_id SERIAL PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    price NUMERIC(10, 2) NOT NULL,  
    stock_quantity INT DEFAULT 0  
);  
  
CREATE TABLE Orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES Customers(customer_id),  
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) DEFAULT 'Pending'  
);
```

```

CREATE TABLE OrderItems (
    order_item_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES Orders(order_id) ON DELETE CASCADE,
    product_id INT REFERENCES Products(product_id),
    quantity INT NOT NULL,
    unit_price NUMERIC(10, 2) NOT NULL
);

-- Триггерлерге арналған қосымша кесте
CREATE TABLE Price_Logs (
    log_id SERIAL PRIMARY KEY,
    product_id INT,
    old_price NUMERIC,
    new_price NUMERIC,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Тестілеу үшін бастапқы деректер (Mock data) енгізу
INSERT INTO Customers (first_name, last_name, email) VALUES
('Асқар', 'Мақсатұлы', 'askar@example.com'),
('Аяжан', 'Ерланқызы', 'ayazhan@example.com');

INSERT INTO Products (product_name, price, stock_quantity) VALUES
('Ноутбук ASUS ROG', 650000.00, 15),
('Смартфон iPhone 15', 550000.00, 30),
('Сымсыз құлаққап AirPods Pro', 120000.00, 50);

INSERT INTO Orders (customer_id, status) VALUES (1, 'Completed'), (2,
'Pending');

INSERT INTO OrderItems (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 1, 650000.00),
(1, 3, 1, 120000.00),
(2, 2, 2, 550000.00);

```

## 2. Процедуралар және қолданушы функциясы

### 2.1. Қолданушы функциялары (4 функция):

```

-- 1. Тауар қалдығын тексеру
CREATE OR REPLACE FUNCTION get_product_stock(p_id INT) RETURNS INT AS $$
DECLARE v_stock INT;
BEGIN SELECT stock_quantity INTO v_stock
FROM Products WHERE product_id = p_id; RETURN v_stock; END;
$$ LANGUAGE plpgsql;

-- 2. Клиенттің жалпы шығынын есептеу
CREATE OR REPLACE FUNCTION get_total_spent(c_id INT) RETURNS NUMERIC AS $$
DECLARE total_spent NUMERIC;
BEGIN
SELECT COALESCE(SUM(quantity * unit_price), 0)
INTO total_spent FROM OrderItems oi
JOIN Orders o ON oi.order_id = o.order_id WHERE o.customer_id = c_id;
RETURN total_spent;
END;
$$ LANGUAGE plpgsql;

-- 3. Жеңілдік мөлшерін есептеу
CREATE OR REPLACE FUNCTION calculate_discount
(price NUMERIC, discount_percent INT) RETURNS NUMERIC AS $$
BEGIN RETURN price - (price * discount_percent / 100); END;
$$ LANGUAGE plpgsql;

-- 4. Клиенттің тапсырыс санын алу
CREATE OR REPLACE FUNCTION count_customer_orders(c_id INT) RETURNS INT AS $$
DECLARE order_count INT;
BEGIN SELECT COUNT(*) INTO order_count FROM Orders WHERE customer_id = c_id; RETURN order_count; END;
$$ LANGUAGE plpgsql;

```

Функцияларды тестілеу нәтижелері:

92SELECT get\_product\_stock(1); -- Күтілетін нәтиже: 20

Data OutputMessagesNotifications

get\_product\_stock

integer

1

20

```
92 SELECT get_product_stock(1); -- Күтілетін нәтиже: 20
93 SELECT get_total_spent(1); -- Күтілетін нәтиже: 770000.00
94 SELECT calculate_discount(100000.00, 15); -- Күтілетін нәтиже: 85000.00
```

Data Output

Messages

Notifications

get\_total\_spent

numeric

1

770000.00

Функцияларға түсіндірме (get\_total\_spent):

"Мәліметтер қорынан дайын есептеулерді алу үшін қолданушы функцияларын жаздым. Мысалы, get\_total\_spent функциясы белгілі бір клиенттің барлық тапсырыстарын іздеп тауып, оның жалпы қанша қаражат жұмсағанын математикалық түрде есептеп, дайын соманы қайтарады. Тестілеу кезінде бұл функцияның дұрыс жұмыс істегенін көруге болады."

2.2. Сақталған процедуралар (4 процедура):

```

-- 1. Жаңа клиент қосу (INSERT)
CREATE OR REPLACE PROCEDURE add_new_customer
(p_first_name VARCHAR, p_last_name VARCHAR, p_email VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN INSERT INTO Customers(first_name, last_name, email)
VALUES (p_first_name, p_last_name, p_email); END;
$$;

-- 2. Қоймадағы тауарды жаңарту (UPDATE)
CREATE OR REPLACE PROCEDURE update_inventory
(p_id INT, added_quantity INT)
LANGUAGE plpgsql AS $$
BEGIN UPDATE Products SET stock_quantity = stock_quantity + added_quantity
WHERE product_id = p_id; END;
$$;

-- 3. Тапсырыс статусын өзгерту (UPDATE)
CREATE OR REPLACE PROCEDURE update_order_status(o_id INT, new_status VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN UPDATE Orders SET status = new_status WHERE order_id = o_id; END;
$$;

-- 4. Тапсырысты жою (DELETE)
CREATE OR REPLACE PROCEDURE delete_order(o_id INT)
LANGUAGE plpgsql AS $$
BEGIN DELETE FROM Orders WHERE order_id = o_id; END;
$$;

```

*Процедураларды тестілеу нәтижелері:*

|                                    |                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------|
| 131                                |                                                                                      |
| 132                                | -- 2-тест: 1-ші тауарға 5 дана қосу                                                  |
| 133                                | CALL update_inventory(1, 5);                                                         |
| 134                                | SELECT stock_quantity FROM Products WHERE product_id = 1; -- 20-тен 25-ға өсуі керек |
| 135                                |                                                                                      |
| Data Output Messages Notifications |                                                                                      |
| Showing rows: 1 to 1               |                                                                                      |
|                                    | stock_quantity<br>integer                                                            |
| 1                                  | 25                                                                                   |

Процедураларға түсіндірме (update\_inventory, т.б.):

"Функциялар есептесе, ал сақталған процедуралар (Stored Procedures) деректер базасына өзгеріс енгізеді (DML). Мен мәлімет қосу, жаңарту және өшіру процедураларын жасадым. Мысалы, update\_inventory процедурасы қоймаға жаңа тауар келгенде, оның санын автоматты түрде қосып, жаңартып отырады."

```

135
136 -- 3-тест: Тапсырыс статусын өзгерту
137 CALL update_order_status(2, 'Shipped');
138 SELECT status FROM Orders WHERE order_id = 2;
139
140

```

| Data Output |                                  | Messages | Notifications |
|-------------|----------------------------------|----------|---------------|
|             | status<br>character varying (20) |          |               |
| 1           | Shipped                          |          |               |

### 3. Триггерлер

#### 3.1. Жол деңгейіндегі (Row-level) және Лог-триггер:

Лог-триггерге түсіндірме (log\_price\_change):

"Деректердің қауіпсіздігі мен тарихын сақтау үшін жол деңгейіндегі (Row-level) лог-триггер жаздым. Егер тауардың бағасы өзгеретін болса, бұл триггер автоматты түрде іске қосылады да, Price\_Logs кестесіне тауардың ескі бағасы мен жаңа бағасын уақытымен бірге жазып қояды. Бұл аудит үшін өте маңызды."

```

147
148 CREATE OR REPLACE FUNCTION log_price_change() RETURNS TRIGGER AS $$
149 BEGIN
150     IF NEW.price <> OLD.price THEN
151         INSERT INTO Price_Logs(product_id, old_price, new_price) VALUES (OLD.product_id, OLD.price, NEW.price);
152     END IF; RETURN NEW;
153 END; $$ LANGUAGE plpgsql;
154
155 CREATE TRIGGER trg_price_change AFTER UPDATE ON Products FOR EACH ROW EXECUTE FUNCTION log_price_change();
156
157 -- ТЕСТІМЕЙ:
158 UPDATE Products SET price = 600000.00 WHERE product_id = 1;
159 SELECT * FROM Price_Logs; -- Лог кестесін тексеру
160

```

| Data Output |                        | Messages              | Notifications                             |
|-------------|------------------------|-----------------------|-------------------------------------------|
|             | log_id<br>[PK] integer | product_id<br>integer | old_price<br>numeric                      |
|             |                        |                       | new_price<br>numeric                      |
|             |                        |                       | changed_at<br>timestamp without time zone |
| 1           | 1                      | 1                     | 650000.00                                 |
|             |                        |                       | 600000.00                                 |
|             |                        |                       | 2026-05-02 07:55:59.869673                |

#### 3.2. Оператор деңгейіндегі (Statement-level) триггер:

```
162
163 CREATE OR REPLACE FUNCTION notify_mass_delete() RETURNS TRIGGER AS $$
164 BEGIN RAISE NOTICE 'Тапсырыстар кестесінен мәлімет өшірілді (Statement Level Trigger)';
165 RETURN NULL; END;
166 $$ LANGUAGE plpgsql;
167
168 CREATE TRIGGER trg_statement_delete AFTER DELETE ON
169 Orders FOR EACH STATEMENT EXECUTE FUNCTION notify_mass_delete();
170
171 -- ТЕСТІЛЕУ:
172 DELETE FROM Orders WHERE status = 'Canceled'; -- Хабарлама шығуын қадағалау
173
174
```

Data Output Messages Notifications

ЗАМЕЧАНИЕ: Тапсырыстар кестесінен мәлімет өшірілді (Statement Level Trigger)  
DELETE 0

Query returned successfully in 52 msec.

### 3.3. INSTEAD OF триггері:

INSTEAD OF триггеріне түсіндірме:

"INSTEAD OF триггері View (Көрініс) арқылы жұмыс істейді. Әдетте күрделі View-ге тікелей мәлімет қосу мүмкін емес. Сондықтан бұл триггер клиенттің аты мен электрондық поштасын алдымен Customers кестесіне салады, содан кейін ғана оған жаңа тапсырыс құрып, Orders кестесіне тіркейді."

```
176 CREATE VIEW Customer_Order_View AS SELECT c.first_name, c.email, o.order_date
177 FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id;
178
179 CREATE OR REPLACE FUNCTION insert_instead_of_view() RETURNS TRIGGER AS $$
180 DECLARE new_cust_id INT;
181 BEGIN
182     INSERT INTO Customers (first_name, email) VALUES (NEW.first_name, NEW.email) ON CONFLICT (email)
183     DO NOTHING RETURNING customer_id INTO new_cust_id;
184     IF new_cust_id IS NULL THEN SELECT customer_id INTO new_cust_id FROM Customers WHERE email = NEW.email; END IF;
185     INSERT INTO Orders (customer_id, order_date) VALUES (new_cust_id, CURRENT_TIMESTAMP);
186     RETURN NEW;
187 END; $$ LANGUAGE plpgsql;
188
189 CREATE TRIGGER trg_instead_of_insert INSTEAD OF INSERT ON Customer_Order_View
190 FOR EACH ROW EXECUTE FUNCTION insert_instead_of_view();
191
192
193 -- ТЕСТІЛЕУ:
194 INSERT INTO Customer_Order_View (first_name, email) VALUES ('Мадина', 'madina@example.com');
195 SELECT * FROM Customers WHERE email = 'madina@example.com'; -- Клиент кестеге түскенін тексеру
196
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

|   | customer_id<br>[PK] integer | first_name<br>character varying (50) | last_name<br>character varying (50) | email<br>character varying (100) |
|---|-----------------------------|--------------------------------------|-------------------------------------|----------------------------------|
| 1 | 4                           | Мадина                               | [null]                              | madina@example.com               |

## 4. Транзакциялар

ACID қағидалары сақталған екі түрлі транзакция тексерілді.

### 4.1. Сәтті орындалатын транзакция (өзгерістерді сақтау):

SQL

```
BEGIN; -- Транзакция бастай [cite: 28]
INSERT INTO Orders (customer_id, status) VALUES (1, 'Processing');
```

```
SAVEPOINT sp_success; -- Сақтау нүктесі
UPDATE Products SET stock_quantity = stock_quantity - 1 WHERE product_id = 3;
COMMIT; -- Транзакцияны бекіту
```

The screenshot shows a SQL IDE with a dark theme. The top pane contains SQL code:
 

```
197
198
199 BEGIN; -- Транзакция бастау [cite: 28]
200 INSERT INTO Orders (customer_id, status) VALUES (1, 'Processing');
201 SAVEPOINT sp_success; -- Сақтау нүктесі
202 UPDATE Products SET stock_quantity = stock_quantity - 1 WHERE product_id = 3;
203 COMMIT; -- Транзакцияны бекіту
204
205
```

 The bottom pane has tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the text 'COMMIT' and 'Query returned successfully in 55 msec.' A green status bar at the bottom right indicates 'Query returned successfully'.

## 4.2. Сәтсіз транзакция (ROLLBACK арқылы кері қайтару):

### SQL

```
BEGIN;
INSERT INTO Orders (customer_id, status) VALUES (2, 'Pending');
SAVEPOINT sp_before_payment; -- Сақтау нүктесі
-- Тауар санын азайтамыз:
UPDATE Products SET stock_quantity = stock_quantity - 500 WHERE product_id = 2;
-- Осы жерде қателік кетті делік, сондықтан кері қайтарамыз:
ROLLBACK TO sp_before_payment; -- Өзгерісті кері қайтару
COMMIT;
```

The screenshot shows a SQL IDE with a dark theme. The top pane contains SQL code:
 

```
205
206
207 BEGIN;
208 INSERT INTO Orders (customer_id, status) VALUES (2, 'Pending');
209 SAVEPOINT sp_before_payment; -- Сақтау нүктесі
210 -- Тауар санын азайтамыз:
211 UPDATE Products SET stock_quantity = stock_quantity - 500 WHERE product_id = 2;
212 -- Осы жерде қателік кетті делік (мысалы, қалдық теріс болып кетті), сондықтан кері қайтарамыз:
213 ROLLBACK TO sp_before_payment; -- Өзгерісті кері қайтару
214 COMMIT;
```

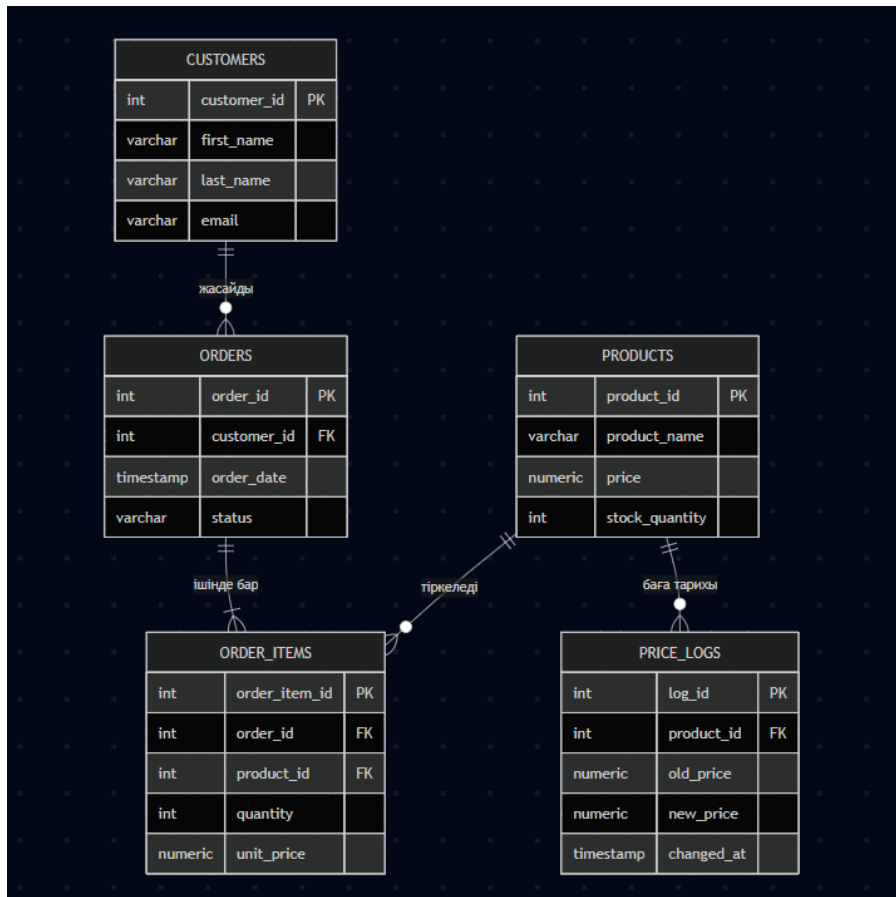
 The bottom pane has tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the text 'COMMIT' and 'Query returned successfully in 38 msec.' A green status bar at the bottom right indicates 'Query returned successfully'.

### Транзакцияларға түсіндірме:

"Жүйеде қателіктер болғанда деректердің бұзылмауын қамтамасыз ету үшін ACID қағидаларымен транзакциялар құрдым. Егер тапсырыс жасалып жатқан кезде тауар қоймада бітіп қалса немесе төлемде қате шықса, ROLLBACK командасы жұмыс істейді де, SAVEPOINT сақтау нүктесіне қайта оралады. Осылайша базаға шала немесе қате мәліметтер жазылмайды. Ал бәрі сәтті болса ғана COMMIT арқылы толық сақталады."

## ER диаграммасы

"Соңында дерекқордың архитектурасын толық түсіну үшін ER диаграмма жасадым. Мұнда 5 кестенің бір-бірімен Primary Key және Foreign Key арқылы қалай байланысып тұрғаны көрсетілген. Кестелер арасындағы "Бірден-көпке" (One-to-Many) қатынастары арқылы жүйенің логикасы толықтай қамтамасыз етілген."



## Қорытынды

СӨЖ жұмысы барысында нормализацияланған мәліметтер базасы құрылды. DML операцияларын тиімді басқару үшін 4 процедура және 4 функция сәтті жазылып тексерілді. Деректерді бақылау және қауіпсіздік үшін 3 түрлі триггер мен ACID қағидаларына негізделген транзакциялар жасалды. Барлық талаптар мен тапсырмалар толық орындалды.